Design and Implementation of AXI4-Lite Interface for BRAM on FPGA

Anirudh Chaturvedi (SR 25850)
Digital Systems Design with FPGAs
MVLSI, Indian Institute of Science, Bangalore

1 Abstract

This report presents the design and implementation of an AXI4-Lite interface to communicate with on-chip BRAM on an FPGA. The system consists of two finite state machines (FSMs), implemented in the AXI4_LITE_MASTER and AXI4_LITE_SLAVE modules. The master FSM interprets commands from the testbench and generates AXI4-Lite signals, while the slave FSM interfaces with a single-port BRAM. The design supports read and write operations through AXI channels and is implemented on a Xilinx Artix-7 FPGA. Functional simulation, post-synthesis simulation, and post-implementation timing simulation are performed to verify correctness. Resource utilization and timing performance are also reported.

2 Introduction

AXI4-Lite is a lightweight version of the AXI protocol used for simple memory-mapped communication between components in FPGA-based systems. In this experiment, an AXI4-Lite interface is designed to control read and write operations to an on-chip Block RAM (BRAM). The design demonstrates the use of finite state machines for protocol implementation and memory interfacing.

The system consists of a top module called `AXI4_LITE_TOP` which instantiates two modules: `AXI4_LITE_MASTER` and `AXI4_LITE_SLAVE`. The master module generates AXI4-Lite signals based on commands received from the testbench, while the slave module interprets these signals and performs memory operations on the BRAM.

3 System Architecture

The overall architecture of the system is shown in Fig. 2. The AXI4-Lite master communicates with the AXI4-Lite slave through five channels: Address Write (AW), Write Data (W), Write Response (B), Address Read (AR), and Read Data (R).

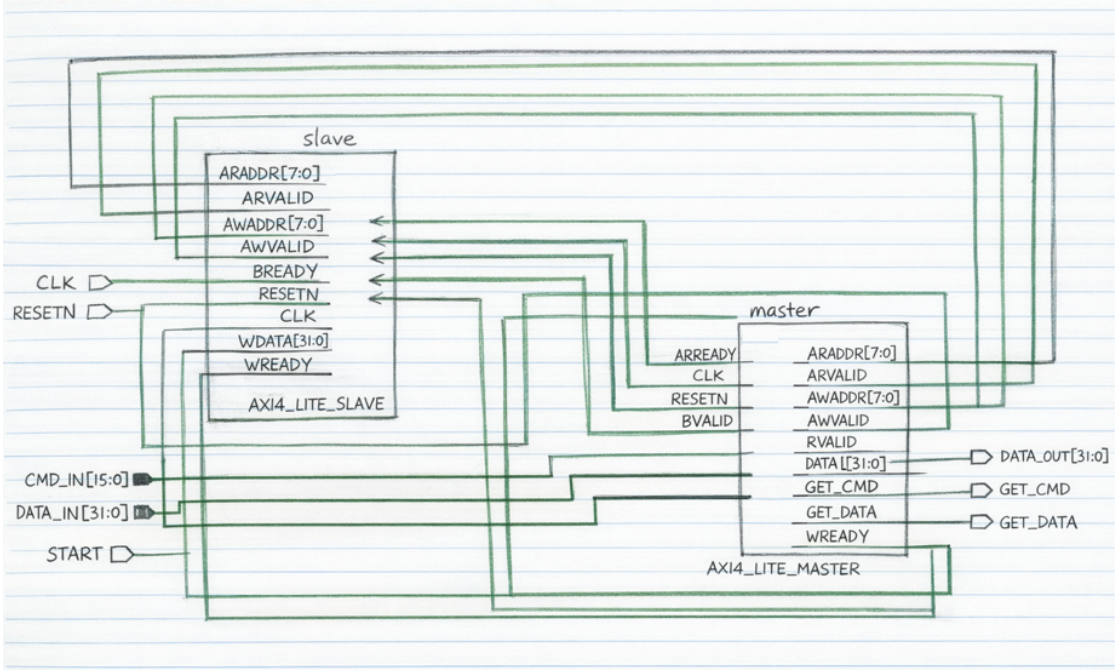


Figure 1: Hand drawn RTL schematic of the AXI4-Lite based system

The command input (`CMD_IN`) consists of 16 bits structured as:

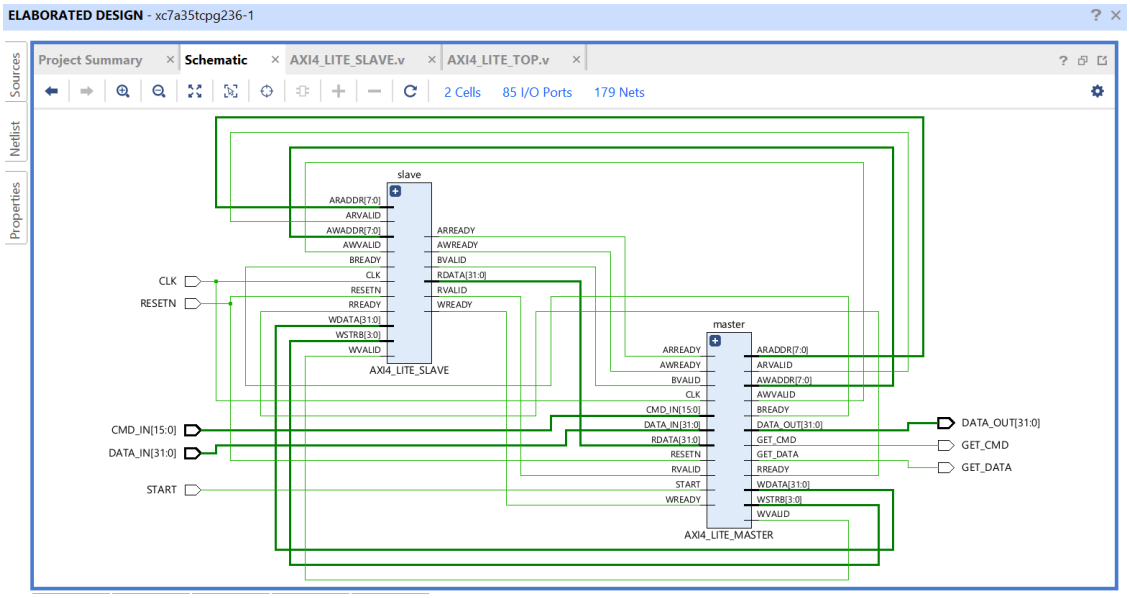


Figure 2: RTL schematic of the AXI4-Lite based system

- 1 bit: Read/Write flag
- 7 bits: Number of words
- 8 bits: Initial memory address

Based on the command type, the master FSM generates appropriate control signals to initiate read or write transactions.

4 Finite State Machine Design

The AXI4-Lite protocol is implemented using FSMs in both the master and slave modules.

4.1 Master FSM

The master FSM generates the AXI signals required for read and write transactions. It transitions through states responsible for:

- Command decoding
- Write address and data transfer
- Write response handling
- Read address generation
- Read data reception

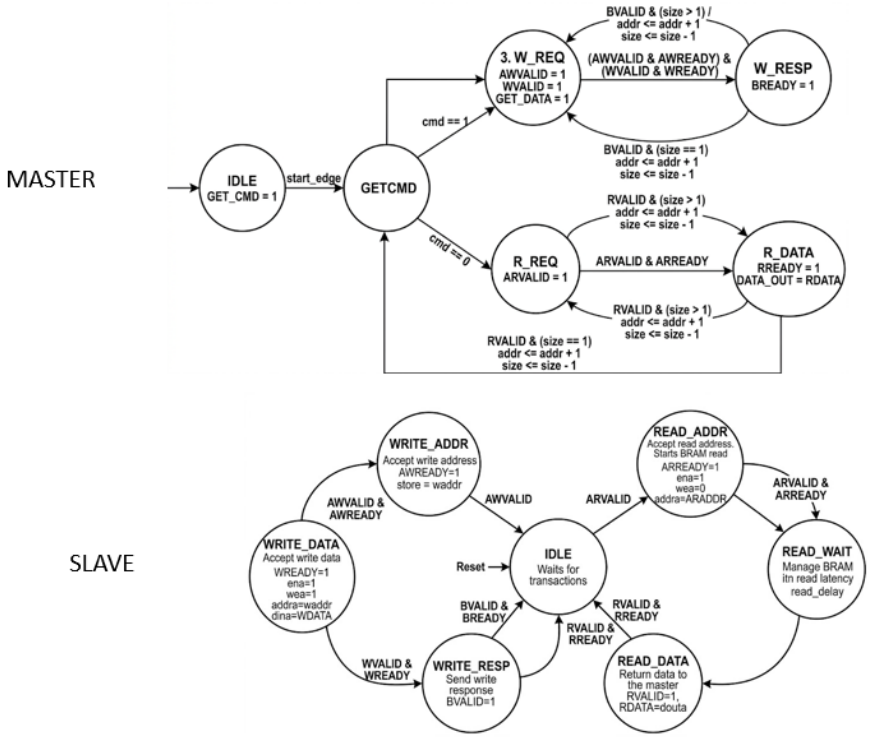


Figure 3: State diagram for AXI4-Lite Master and Slave FSMs

4.2 Slave FSM

The slave FSM interprets the signals from the master and performs the required operations on the BRAM. It handles address decoding, memory access, and response generation.

5 Implementation

The design was implemented on a Xilinx Artix-7 FPGA. The BRAM is configured as a single-port RAM with the following parameters:

- Word width: 32 bits
- Depth: 256 locations
- Address width: 8 bits

The slave module contains the instantiated BRAM and controls read/write operations based on the AXI protocol signals.

6 Simulation Results

6.1 Behavioral Simulation

Behavioral simulation verifies the correctness of the design before synthesis.

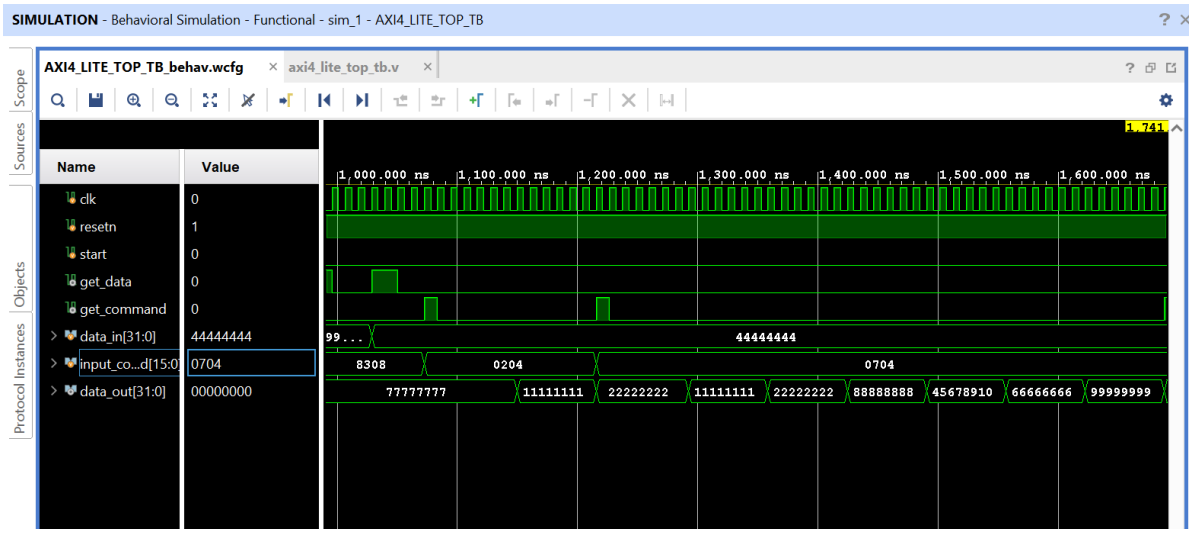


Figure 4: Behavioral simulation waveform

6.2 Post-Synthesis Functional Simulation

The synthesized netlist was simulated to verify functional correctness.

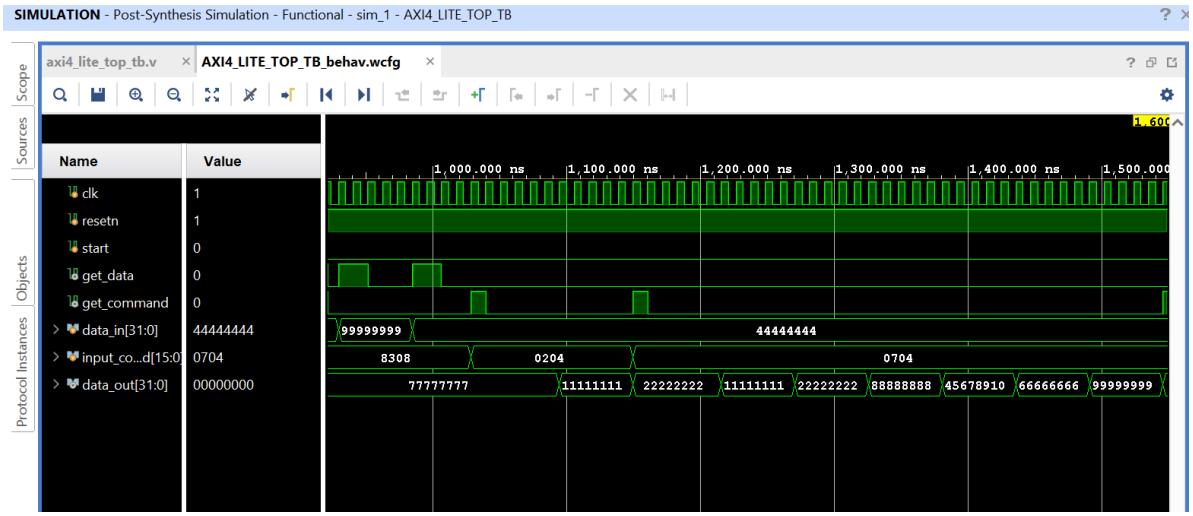


Figure 5: Post-synthesis functional simulation

6.3 Post-Synthesis Timing Simulation

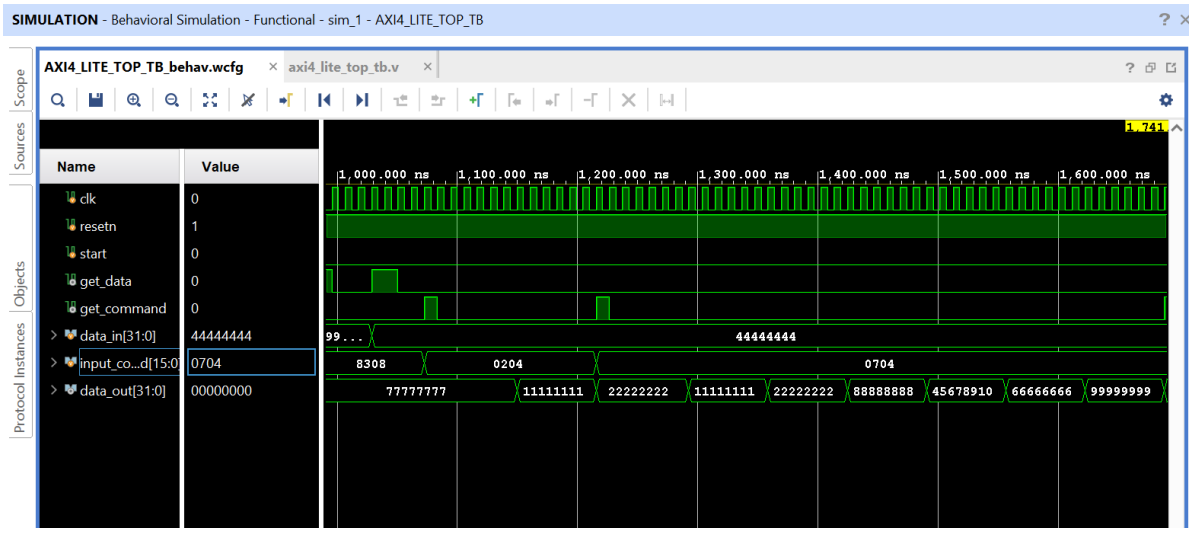


Figure 6: Post-synthesis timing simulation

6.4 Post-Implementation Simulation

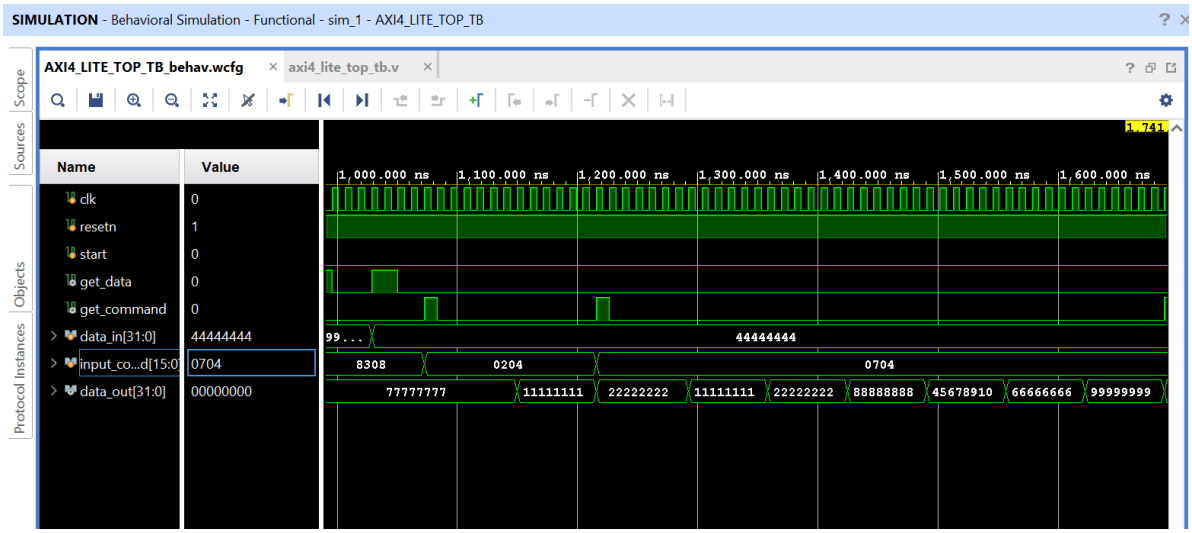


Figure 7: Post-implementation functional simulation

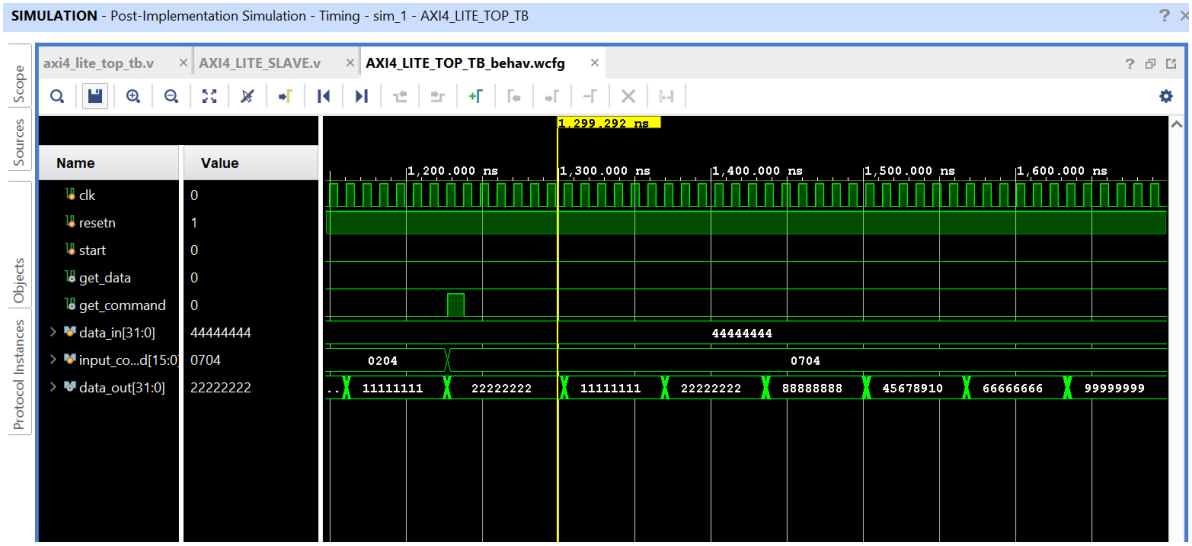


Figure 8: Post-implementation timing simulation

7 Resource Utilization

The FPGA resource utilization for the design is shown in Fig. 9.

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	61	20800	0.29
FF	200	41600	0.48
BRAM	0.50	50	1.00
IO	85	106	80.19
BUFG	1	32	3.13

Figure 9: Resource utilization summary

The design uses a small number of LUTs, registers, and BRAM resources, demonstrating the efficiency of the AXI4-Lite interface implementation.

8 Timing Performance

Timing analysis was performed after implementation to determine the maximum operating frequency.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5.878 ns	Worst Hold Slack (WHS):	0.063 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	400	Total Number of Endpoints:	400	Total Number of Endpoints:	203

All user specified timing constraints are met.

Figure 10: Timing analysis summary

The timing report indicates that the design meets the required timing constraints and operates reliably at the specified clock frequency.

9 Power Analysis

Power consumption of the design is shown in Fig. 11.

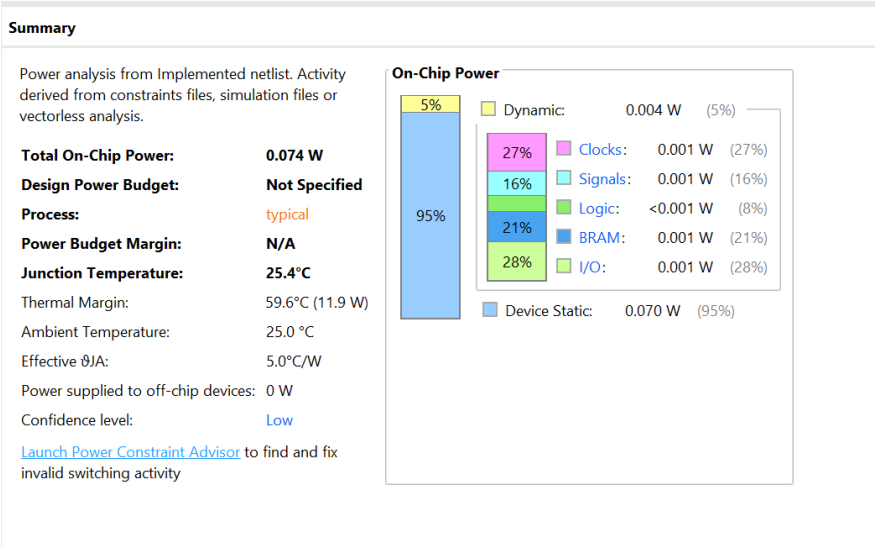


Figure 11: Power estimation

The total power consumption is low due to the lightweight nature of the AXI4-Lite interface and the limited resource utilization.

10 Conclusion

In this work, an AXI4-Lite interface was successfully implemented to control read and write operations to on-chip BRAM. The design used separate FSMs for the master and slave modules to implement the AXI protocol. Functional and timing simulations verified correct operation of the system. The design was implemented on a Xilinx Artix-7 FPGA and demonstrated efficient resource utilization and satisfactory timing performance.

References

[1] ARM Ltd., *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022D, 2011.

[2] Xilinx Inc., *7 Series FPGAs Overview*, Xilinx Product Specification DS180, 2023.

[3] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3rd ed., McGraw-Hill, 2013.

[4] S. Trimberger, "Three Ages of FPGAs: A Retrospective on the First Thirty Years of FPGA Technology," *Proceedings of the IEEE*, vol. 103, no. 3, pp. 318–331, 2015.

[5] D. Gisselquist, "Building an AXI-Lite Slave," *ZipCPU Blog*, 2019.

[6] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Prentice Hall, 2003.